# INFORME ACTUALIZADO - SESIONES 10 Y 11

Autenticación JWT, PharmaSoft y persistencia SQLDelight offline-first

| | | | |
|---|---|---|---|
| **ESTUDIANTE** | Guillen Mendoza Alexander | **DOCENTE** | David Reyna |
| **ASIGNATURA** | Desarrollo de Aplicaciones Móviles | **FECHA DE ACTUALIZACIÓN** | 25 de octubre de 2026 |
| **PROYECTO** | PharmaMobile_Practica + PharmaSoft | **RAMA** | feature/offline-first-guillen |

## 1. Resumen ejecutivo

Se integró una sesión JWT de PharmaSoft en la app Kotlin Multiplatform. La solución incluye login, recuperación y cierre de sesión, token cifrado por plataforma, autenticación automática en Ktor, tratamiento distinto de 401 y 403, y secciones REST para productos y clientes. En el servidor se levantaron Oracle Free y Spring Boot mediante Docker Compose, con esquema y datos semilla aplicados por Flyway.

**Resultado comprobado: Oracle saludable, cuatro migraciones aplicadas, autenticación y autorización HTTP correctas, CRUD real de un producto temporal (201/200/204), pruebas Maven completas sin fallos y 28 pruebas móviles de host correctas. La ejecución visual de la app contra el backend y la ejecución iOS aún requieren evidencia adicional.**

## 2. Objetivo y contexto

La Guía Práctica N.º 10 solicita integrar el login JWT, adjuntar el token con el plugin Auth de Ktor, protegerlo en Android e iOS, ocultar el token en los registros y distinguir sesión expirada de falta de permisos. Este informe corresponde a la implementación realizada sobre la rama móvil que ya contenía el trabajo de sesiones anteriores. Se conservaron el inventario local demostrativo y el catálogo público de Platzi; Pedidos sigue siendo una demostración, no un CRUD REST.

Los insumos recibidos fueron la guía PDF, una colección Postman de 63 solicitudes de seguridad y un RAR con migraciones V2/V3. El RAR no era una base móvil; su V2 contenía una línea SQL inválida, por lo que se empleó la migración corregida del repositorio PharmaSoft. No se copiaron credenciales ni secretos al código fuente.

## 3. Arquitectura de la solución

| CAPA | RESPONSABILIDAD IMPLEMENTADA |
|---|---|
| Dominio | Modelos de sesión y errores, contratos de repositorio y casos de uso; sin Ktor, Android ni UIKit. |
| Datos comunes | DTO, cliente Ktor, Auth Bearer, traducción de errores y repositorios REST de autenticación, productos y clientes. |
| Plataformas | AlmacenSeguroAndroid con Android Keystore/EncryptedSharedPreferences; AlmacenSeguroIos con Keychain; URL base por expect/actual. |
| Presentación | ViewModels, LoginScreen, pantallas de PharmaSoft, estados de carga/error, navegación y acciones visibles según permisos. |
| Servidor | Spring Security/JWT, controladores REST, Oracle Free y migraciones Flyway V1-V4. |

## 4. Funcionalidad móvil implementada

Inicio y recuperación: si no existe token se presenta el login. Con token guardado, la app consulta /auth/me para recuperar identidad y permisos. Un fallo temporal de red permite reintentar sin borrar automáticamente la credencial.

Login: POST /api/v1/auth/login envía usuario y contraseña; el repositorio guarda el JWT en AlmacenSeguro, limpia la caché del proveedor Bearer y descarta la contraseña del estado de interfaz. El DTO de /auth/me se ajustó al campo real empleadoNombre.

Solicitudes protegidas: un cliente Ktor exclusivo de PharmaSoft instala ContentNegotiation, DefaultRequest, Auth Bearer, HttpTimeout y Logging. El token se envía desde la primera solicitud a /api/v1/ excepto al login. No se escribe Authorization manualmente en los composables. El cliente público de Platzi permanece separado.

Seguridad y errores: el logging de depuración usa cabeceras y sanea Authorization como ***; en otras compilaciones está apagado. Un 401 protegido borra la sesión y vuelve al login; un 401 del login significa credenciales inválidas. Un 403 muestra falta de permiso sin cerrar la sesión. El cierre voluntario elimina el token y limpia la caché Bearer.

Permisos y CRUD: CATALOGO_LEER/ESCRIBIR y CLIENTES_LEER/ESCRIBIR determinan qué secciones y botones muestra la interfaz. Las pantallas REST permiten listar, registrar, actualizar y eliminar productos y clientes. La restricción visual no sustituye la autorización del backend.

## 5. Implementación del servidor y Oracle

Docker Compose ejecuta una imagen oficial de Oracle Free y el backend Spring Boot. Oracle escucha en el puerto local 1522; la API, en el 8080. Los puertos se publican solo en 127.0.0.1. El volumen oracle_data conserva los archivos de la base entre reinicios. Un script idempotente crea PHARMADB al arranque si no existe y aplica sus permisos; se fijó el formato LF del script para evitar errores de Bash en Windows.

El perfil container configura JDBC, Hibernate y Flyway. V1 crea el esquema inicial; V2 agrega empleados, usuarios y roles; V3 siembra usuarios de desarrollo con hashes BCrypt; V4 agrega tres categorías. Se corrigió la configuración CORS requerida por Spring Security. Las contraseñas y el secreto JWT están en un archivo .env ignorado por Git, nunca en el PDF ni en el repositorio.

| MIGRACIÓN | RESULTADO COMPROBADO |
|---|---|
| V1 | Esquema inicial del negocio aplicado por Flyway. |
| V2 | Tablas y relaciones de seguridad aplicadas con la versión SQL corregida. |
| V3 | Usuarios y roles de desarrollo disponibles para pruebas JWT. |
| V4 | Tres categorías recuperadas desde la API real. |

## 6. Verificación ejecutada

| PRUEBA | EVIDENCIA O RESPUESTA | ESTADO |
|---|---|---|
| Oracle / PHARMADB | Contenedor saludable y conexión SQL real | Correcto |
| Flyway | Cuatro migraciones conservadas tras recrear Oracle | Correcto |
| GET /api/health | 200 | Correcto |
| GET productos sin token | 401 | Correcto |
| Login admin y /auth/me | Token emitido; /auth/me devuelve admin | Correcto |
| GET productos con admin | 200 | Correcto |
| GET categorías | Tres registros | Correcto |
| GET clientes con almacen | 403; la sesión sigue autorizada para catálogo | Correcto |
| Producto temporal | POST 201, GET 200, DELETE 204 | Correcto |
| Pruebas Maven | mvnw.cmd test contra Oracle | Sin fallos |
| Pruebas y compilación móvil | 28 pruebas host; APK debug y Kotlin iOS compilados | Correcto |

La prueba del producto usó un registro temporal que se eliminó al terminar. Tras recrear Oracle sin borrar el volumen, siguieron presentes las migraciones, las tres categorías y el login. El flujo real fue comprobado por HTTP desde el equipo anfitrión; no equivale a haberlo recorrido manualmente en la pantalla móvil.

## 7. Criterios de la guía: estado honesto

| CRITERIO | ESTADO |
|---|---|
| 1. Login JWT | Implementado; verificado por HTTP y pruebas móviles |
| 2. Almacén por plataforma | Implementado; compilado, sin prueba visual iOS |
| 3-4. Auth y envío inicial | Implementado; cubierto con MockEngine |
| 5. Log saneado | Implementado y probado; falta captura de Logcat |
| 6. Persistencia tras reapertura | Prueba automatizada; falta ciclo manual de app |
| 7-9. 401, 403 y permisos UI | Probados con mocks; 401/403 reales del servidor comprobados |
| 10. Android e iOS en ejecución | Pendiente de evidencia funcional en ambos |

## 8. Publicación y reproducibilidad

La aplicación se publicó en master y en feature/auth-jwt-guillen del repositorio del estudiante. El backend se publicó en backend/oracle-jwt del mismo repositorio, como rama independiente con sus propios archivos e historial. No debe fusionarse con master como si fuera una característica de la app: es un proyecto distinto. El remoto original del backend, perteneciente al docente, no se modificó.

| CONTENIDO | RAMA Y REVISIÓN PUBLICADA |
|---|---|
| Aplicación | master y feature/auth-jwt-guillen - 5be2535 |
| Servidor | backend/oracle-jwt - a88a22f |

Repositorio: github.com/alexgm-A11/PharmaMobile_Practica

Documentación técnica: README.md y docs/S10_AUTENTICACION_JWT.md en master; README.md del backend en backend/oracle-jwt.

## 9. Limitaciones y siguientes verificaciones

Postman: se analizaron las 63 solicitudes, pero no se ejecutó la colección completa. Sí se verificaron directamente los casos fundamentales de 401, 200 y 403. Falta guardar capturas exigidas por la guía.

Android: el APK compiló y se había instalado/abierto en Pixel 6 sin fallo de arranque. Durante la verificación final no había emulador conectado; faltan el recorrido real de login, CRUD, reapertura con sesión guardada, 403 visible y captura del log con Authorization saneada.

iOS: compiló el código Kotlin compartido para el simulador, pero Windows no ejecuta iosApp ni demuestra el comportamiento del Keychain. La prueba y capturas iOS requieren macOS y Xcode.

Producción: sustituir usuarios semilla y contraseñas de desarrollo, utilizar HTTPS y revisar la estrategia de almacenamiento Android, pues EncryptedSharedPreferences está obsoleta. El tráfico HTTP claro se permite únicamente en la compilación Android debug.

## 10. Conclusión

La integración JWT y la infraestructura Oracle quedaron implementadas y verificadas en las capas correspondientes. El backend inicia, migra y conserva datos; el acceso protegido y un ciclo real de CRUD responden correctamente. La app dispone del flujo de sesión y CRUD autenticado, con pruebas automatizadas satisfactorias. Para presentar el criterio de funcionamiento en ambas plataformas como completo aún se necesitan las ejecuciones y capturas manuales indicadas.

Fuentes de alcance: Guía Práctica de Laboratorio N.º 10 (13/10/2026), colección Postman de seguridad y migraciones recibidas; código y documentación de las revisiones Git indicadas. Los resultados de prueba corresponden a la verificación local del 13/10/2026.

# ANEXO - ACTIVIDAD AUTÓNOMA N.º 10

Validación de expiración, almacenamiento y cierre de sesión

## Alcance de este anexo

La actividad se incorporó al informe a partir de la guía y de la revisión del código y de los resultados disponibles. No se modificó el código de la app ni del backend para preparar este anexo. Se distingue entre comportamiento verificado en pruebas o por HTTP, comportamiento identificado por inspección del código y escenarios que todavía requieren ejecución y captura en dispositivo. No se presentan resultados esperados como si fueran observaciones reales.

## Producto 1. Bitácora de los seis escenarios de token

La guía exige fecha/hora, plataforma, pasos, código HTTP, mensaje literal, pantalla observada, captura y conclusión propia por escenario. El cuadro resume la evidencia disponible; las capturas y los campos que no fueron registrados quedan expresamente pendientes.

| N.º / escenario | Respuesta esperada según guía y backend | Verificación disponible y conclusión |
|---|---|---|
| 1. Token válido | HTTP 200 en una operación autorizada. | HTTP 200 confirmado al consultar productos con admin y token válido. No se registró mensaje literal (respuesta paginada) ni pantalla/captura Android/iOS. Conclusión: servidor acepta el token válido; falta evidencia en la app. |
| 2. Token expirado | HTTP 401. Mensaje del servidor: El token ha expirado. Inicie sesión nuevamente. | No ejecutado por espera de expiración real. La app trata el 401 protegido como sesión expirada y el mensaje UI previsto es La sesión expiró. Inicia sesión de nuevo.; hay prueba MockEngine del evento 401. No hay respuesta/captura real de expiración. |
| 3. Token alterado | HTTP 401. Mensaje esperado: El token no es válido. | No ejecutado con token alterado. El filtro del backend define ese mensaje; la app convierte cualquier 401 protegido en sesión expirada. Conclusión: falta probar si la pantalla informa de manera adecuada el token alterado. |
| 4. Sin cabecera | HTTP 401. Mensaje esperado: Debe iniciar sesión para acceder a este recurso. | HTTP 401 real confirmado en GET productos sin token. El cuerpo literal y una solicitud posterior al logout no se guardaron. Conclusión: servidor rechaza el acceso anónimo; falta captura del mensaje y UI. |
| 5. Prefijo bearer minúsculo | HTTP 401, tratado como petición anónima. | No ejecutado. El filtro exige el prefijo exacto Bearer y omite otro formato. Sin respuesta HTTP capturada; conclusión basada en revisión del código. |
| 6. Usuario deshabilitado | HTTP 401. Mensaje esperado: El usuario está deshabilitado. | No ejecutado para no cambiar el estado del usuario semilla durante esta actualización documental. El filtro consulta si el usuario está habilitado en cada petición. Falta provocar el caso y registrar respuesta y pantalla. |

Fecha/hora y plataforma de las solicitudes directas: 13/10/2026, equipo anfitrión Windows. Las observaciones de token válido y ausencia de token son HTTP al backend; no son capturas de Android/iOS. Los otros cuatro escenarios no se ejecutaron. La guía pide evidencias reales y mensajes del campo message; este anexo no inventa esos datos.

## Producto 2. Evidencia de almacenamiento seguro

| Comprobación solicitada | Implementación o evidencia disponible | Estado para la entrega |
|---|---|---|
| Android: archivo cifrado en shared_prefs | AlmacenSeguroAndroid usa EncryptedSharedPreferences con llave AES256_GCM protegida por MasterKey/Android Keystore; archivo sesion_segura, clave lógica jwt. | Código revisado; no se capturó Device Explorer ni se inspeccionó un dispositivo autenticado. Pendiente captura que demuestre que el valor no está legible. |
| Authorization saneado en Logcat | Logging sanea la cabecera Authorization. Una prueba MockEngine verifica el saneamiento del registro. | Prueba automatizada disponible; falta captura real de Logcat con Bearer ***. |
| Sesión tras cerrar y reabrir | El arranque consulta AlmacenSeguro y llama /auth/me; una prueba común restaura un token desde un almacén en memoria simulado. | La prueba simula almacenamiento y no prueba persistencia Android física. Falta iniciar sesión, forzar cierre y reabrir la app con captura. |
| iOS: servicio y cuenta Keychain | El código usa servicio pe.edu.upeu.pharmamobile.sesion, cuenta jwt y accesibilidad AfterFirstUnlockThisDeviceOnly. | Implementación inspeccionada y source set compilado; no hay Mac/Xcode disponible para verificar lectura/escritura ni captura iOS. Debe declararse esa limitación. |

## Producto 3. Cierre de sesión completo

| Efecto requerido | Estado observado en código/pruebas | Conclusión documental |
|---|---|---|
| 1. Borrar token del almacén | cerrarSesion llama borrarToken(); prueba común comprueba token nulo en almacén simulado. | Implementado y cubierto en prueba común; falta inspección física del almacén y captura. |
| 2. Olvidar token en caché HTTP | cerrarSesion ejecuta clearToken() del BearerAuthProvider; el login también invalida la caché tras guardar token. | Implementado por código; falta probar la siguiente petición real sin cabecera y registrar el 401. |
| 3. Limpiar estado de interfaz | El ViewModel borra sesión, clave y error. La propiedad usuario se conserva en el estado; el campo de login puede mostrar el usuario anterior. | Hallazgo pendiente: no cumple por completo el requisito de no mostrar el usuario anterior. No se corrigió porque el alcance solicitado fue solo documental. |
| 4. Impedir volver a pantalla autenticada | La app cambia a LoginScreen cuando la sesión es nula; la pantalla autenticada sale de composición. La navegación se mantiene en estado Compose local y no se documentó una prueba del botón Atrás del sistema. | El código descarta el contenido autenticado; falta verificar manualmente que Atrás no restaure la pantalla previa. |

## Producto 4. Matriz de permisos por rol

El API real comprobó admin en productos y /auth/me, y almacén recibiendo 403 en clientes. El resto de combinaciones se consigna como resultado esperado de la práctica/servidor y oferta derivada de los permisos en el código, no como ejecución real. La app móvil no tiene ventas REST ni una sección de reportes: Pedidos es una pantalla demostrativa filtrada por VENTAS_LEER.

| Operación | Rol | HTTP esperado | Interfaz según código | HTTP real capturado |
|---|---|---|---|---|
| Listar productos | admin | 200 | Visible | 200 |
| Listar productos | vendedor | 200 | Visible | No ejecutado |
| Listar productos | almacen | 200 | Visible | No ejecutado |
| Registrar producto | admin | 201 | Visible | 201 (temporal) |
| Registrar producto | vendedor | 403 | Oculta | No ejecutado |
| Registrar producto | almacen | 201 | Visible | No ejecutado |
| Listar clientes | admin | 200 | Visible | No ejecutado |
| Listar clientes | vendedor | 200 | Visible | No ejecutado |
| Listar clientes | almacen | 403 | Oculta | 403 |
| Registrar venta | admin | 201 | Pedidos demo; sin venta REST | No ejecutado |
| Registrar venta | vendedor | 201 | Pedidos demo; sin venta REST | No ejecutado |
| Registrar venta | almacen | 403 | Sección Pedidos oculta | No ejecutado |
| Ver reportes | admin | 200 | Sin pantalla de reportes | No ejecutado |
| Ver reportes | vendedor | 403 | Sin pantalla de reportes | No ejecutado |
| Ver reportes | almacen | 403 | Sin pantalla de reportes | No ejecutado |
| Consultar /auth/me | admin | 200 | Usuario/rol en sesión | 200 |
| Consultar /auth/me | vendedor | 200 | Usuario/rol en sesión | No ejecutado |
| Consultar /auth/me | almacen | 200 | Usuario/rol en sesión | No ejecutado |

## Evaluación frente a la actividad

La base de sesión, el borrado del token, la limpieza de caché Bearer y la visualización por permisos existen en el proyecto. Hay pruebas automatizadas para 401/403 y para ocultar clientes/ventas a almacén. Sin embargo, esta actualización no incluye bitácora experimental completa, capturas por escenario, evidencia física de almacenamiento, recorrido manual de logout, ni una matriz HTTP completa de 18 celdas. Además, se registra que el nombre de usuario permanece en el campo de login tras el logout y que ventas/reportes no tienen integración REST en esta rama.

**Estado de la entrega: informe complementado con la especificación y con el estado comprobable del proyecto. Los resultados marcados 'No ejecutado' y las capturas pendientes no deben presentarse como experimentos realizados. Fecha límite indicada por la guía: lunes 19 de octubre de 2026 a las 23:59.**

# ANEXO - SESIÓN 11: PERSISTENCIA OFFLINE-FIRST

Actualización documental solicitada según la Guía Práctica de Laboratorio N.º 11

## 1. Alcance y criterio de honestidad

Este anexo integra la Guía Práctica N.º 11 y la Actividad Autónoma N.º 11 con el código realizado en la rama feature/offline-first-guillen. Se añadió persistencia local SQLDelight, sincronización con PharmaSoft, visualización de antigüedad de datos y seis pruebas comunes. Los requisitos de PDF se distinguen de las observaciones y pruebas realmente ejecutadas.

**Estado: los seis casos automatizados pasan y compilan Android y el source set compartido de iOS. No se completaron los recorridos manuales con modo avión, las capturas Android/iOS ni la comprobación real de sincronización en Swagger: no había emulador conectado y Docker Desktop estaba apagado. Esos puntos quedan como evidencia pendiente, no como resultado atribuido.**

## 2. Requisitos de la guía y estado actual

| REQUISITO DE SESIÓN 11 | QUÉ DEBE EXISTIR / CÓMO SE COMPRUEBA | ESTADO DE LA RAMA REVISADA |
|---|---|---|
| Esquema SQLDelight | Plugin 2.1.0, PharmaDb, tabla producto con marca pendiente y metadato de sincronización. | Implementado en Producto.sq. |
| Drivers por plataforma | Android, iOS y JDBC en pruebas; registro en Koin. | Implementados; compilación Android e iOS shared correcta. |
| Fuente de verdad local | Pantalla observa la base local mediante Flow. | Implementado en ProductoRepository, ProductoLocal y ProductoViewModel. |
| Crear sin conexión | Guardar localmente con id negativo y marca pendiente. | Implementado y cubierto por prueba automatizada. |
| Sincronización | Enviar pendientes primero; luego descargar y reemplazar filas sincronizadas en transacción. | Implementado en ProductoRepositorioOffline; cubierto por pruebas. |
| ViewModel | Observar una vez, informar red y ofrecer sincronización manual. | Implementado; pruebas automáticas de presentación no incluidas. |
| Pruebas commonTest | Seis casos, base JDBC independiente y sin red. | Seis pruebas nuevas pasan; suite total 37. |
| Evidencia manual | Modo avión, reconexión, Swagger, Android/iOS. | Pendiente por falta de emulador, Docker activo y macOS/Xcode. |

## 3. Implementación offline-first realizada

El contrato ProductoRepository ahora ofrece observar(): Flow>, sincronizar() y última sincronización. SQLDelight genera PharmaDb a partir de Producto.sq. ProductoLocal traduce filas a dominio, genera ids negativos para altas offline, obtiene pendientes y reemplaza sincronizados en transacción. Android usa AndroidSqliteDriver y iOS NativeSqliteDriver, registrados junto a PharmaDb mediante Koin. El driver SQLite JDBC permite pruebas en memoria.

ProductoRepositorioOffline registra local primero, sube las filas pendientes mediante PharmaSoftProductosRemotos y después baja la lista. El borrado solo alcanza filas sincronizadas, así que la descarga no borra pendientes. Al subir una fila la reemplaza con el id real del backend. ProductoViewModel recoge el Flow sin recargar manualmente tras cada alta; la pantalla indica pendientes, aviso de conexión y tiempo relativo desde la última sincronización correcta. Sin marca previa, muestra “Nunca sincronizado”.

El perfil mínimo de usuario y permisos se conserva cifrado en EncryptedSharedPreferences/Keychain junto al token. Si no hay red para `/auth/me`, la app ofrece modo sin conexión, mantiene acceso al inventario y limita el menú a Inicio/Productos. Las altas locales pueden quedar pendientes; la autorización remota sigue siendo responsabilidad de PharmaSoft. Cuando una petición recibe un 401 real, se borran token y perfil cacheado y se requiere nuevo login.

Para el endpoint del backend se usa la categoría id 1 (Analgesicos, sembrada por V4), ya que el formulario actual no ofrece categoría. Si se cambia la semilla del backend, debe ajustarse este adaptador o añadirse selección de categoría. La sincronización remota requiere JWT vigente; si /auth/me no puede validarse por falta de red, el perfil cifrado habilita únicamente el modo local, según el escenario 5.

## 4. Plan de verificación y evidencia todavía necesaria

| ETAPA | PROCEDIMIENTO SOLICITADO | EVIDENCIA QUE DEBE ADJUNTARSE |
|---|---|---|
| Suite commonTest | Se ejecutaron seis escenarios offline con SQLite en memoria y Fake remoto, más pruebas de sesión offline. | Pasan 37 pruebas host en total. Salida Gradle resumida en el anexo de actividad autónoma. |
| Modo avión Android | La guía pide reabrir, leer datos previos, registrar y observar marca pendiente. | No ejecutado manualmente: no había emulador conectado. La prueba automatizada cubre registro sin red; captura pendiente. |
| Reconexión y Swagger | La guía pide ver el registro en PharmaSoft con id positivo. | Automatizado con Fake remoto; Swagger real pendiente. Docker Desktop no estaba activo. |
| iOS | Abrir listado sin red y registrar desde driver nativo. | El source set iosSimulatorArm64 compila; ejecución/captura requieren macOS y Xcode. |
| README y rama | Documentar y actualizar rama individual. | README y docs actualizados; rama creada localmente. No se hizo push a GitHub. |

El escenario 5 cruza autenticación y offline-first. Sin conexión, un JWT que no se puede validar no se trata como válido para PharmaSoft: el perfil cifrado solo permite acceso local y acota la navegación a Productos/Inicio. Al volver la red, si el servidor responde 401 se borra el perfil y se pide login; después del login se pueden reintentar las filas pendientes. Los permisos cacheados pueden quedar obsoletos hasta esa validación, por lo que no se habilitan pantallas remotas en modo offline.

La implementación se organizó en la rama local feature/offline-first-guillen con dos commits funcionales: 71d2f2a (persistencia SQLDelight y sincronización) y ee75e41 (sesión offline, UI y pruebas). No se publicó la rama en GitHub.

## 5. Conclusión actualizada

Fuente para este anexo: Guía Práctica de Laboratorio N.º 11 y Actividad Autónoma N.º 11; revisión del código y ejecución Gradle de feature/offline-first-guillen, 25/10/2026.

# ANEXO - PRODUCTOS DE LA ACTIVIDAD AUTÓNOMA N.º 11

Pruebas, bitácora, antigüedad de datos y guion de sustentación

## Producto 1. Suite de seis pruebas commonTest

| N.º | CASO | RESULTADO |
|---|---|---|
| 1 | Sincronización con red: datos remotos escritos y emitidos por Flow. | Pasa |
| 2 | Registro sin red: queda en local con id negativo y marca pendiente; Flow lo muestra. | Pasa |
| 3 | Envío al reconectar: se registra una vez, toma id servidor y deja de estar pendiente. | Pasa |
| 4 | Bajada remota no borra una fila pendiente. | Pasa |
| 5 | Ids negativos locales conviven sin colisión con ids positivos remotos. | Pasa |
| 6 | Dos sincronizaciones consecutivas no duplican filas. | Pasa |

Cada prueba crea su propio PharmaDb con JdbcSqliteDriver.IN_MEMORY y usa FakeProductosRemotos, sin internet, espera ni emulador. La suite también verifica la restauración de perfil al fallar la red, la limpieza de perfil al salir/recibir 401 y el menú local restringido. Comando ejecutado: ./gradlew.bat :shared:testAndroidHostTest :androidApp:assembleDebug :shared:compileKotlinIosSimulatorArm64. Resultado literal final: BUILD SUCCESSFUL; 37 pruebas host completadas. También se generó androidApp-debug.apk y compiló el código compartido iOS. La compilación iOS no ejecuta Simulator ni prueba la interfaz en un dispositivo.

## Producto 2. Bitácora de escenarios offline

| ESCENARIO | ESTADO Y CONCLUSIÓN | EVIDENCIA |
|---|---|---|
| 1. Apertura sin red con datos previos | El fallback de perfil y la lectura local tienen pruebas separadas. Falta el recorrido combinado en modo avión. | Captura pendiente |
| 2. Registro sin red | La prueba común confirma guardado pendiente e inmediato en Flow. Recorrido manual y aspecto de UI aún no observados. | Prueba automática; captura pendiente |
| 3. Reconexión y envío | Prueba Fake confirma id positivo y fila no pendiente. La guía exige además probar con API real. | Swagger real pendiente; Docker apagado |
| 4. Primera apertura sin red | Pruebas usan una base vacía nueva, pero no se abrió la app en modo avión. Debe comprobarse que presenta estado vacío y no error. | Captura pendiente |
| 5. JWT expirado sin red | No ejecutado visualmente. El fallo de red usa perfil offline cifrado; un 401 real borra el perfil y exige login. El modo permite acceso local, no acceso remoto. Falta probar/capturar el recorrido. | Captura y ensayo pendientes |

No había emulador conectado; Docker Desktop tampoco estaba activo. Por ello, no se inventan horas de ejecución, texto observado ni capturas. Para cerrar esta parte, repetir cada escenario en Android; los de iOS requieren Mac/Xcode. En el escenario 3, confirmar en Swagger el id positivo: verlo en la app no basta.

## Producto 3. Antigüedad de datos

Implementado: SQLDelight guarda el instante de sincronización completa exitosa; el ViewModel lo presenta como “Actualizado hace menos de un minuto”, minutos, horas o días. Si no existe marca, muestra “Nunca sincronizado”. Las capturas de una sincronización reciente y de más de 24 horas quedan pendientes porque no fue posible ejecutar el recorrido visual.

## Producto 4. Guion de sustentación (8 minutos)

| TIEMPO | CRITERIO / DEMOSTRACIÓN | PREPARACIÓN |
|---|---|---|
| 0-1 min | JWT: login correcto como admin y login incorrecto. | Cerrar sesión antes de empezar. |
| 1-3 min | REST + CRUD: listar, registrar y actualizar producto con admin. | Oracle/API activos; categoría y permiso listos. |
| 3-5 min | 403 al intentar escribir como almacen; error de validación de campo como admin. | Cuenta almacen y formulario inválido preparados. |
| 5-7 min | Offline-first con admin: modo avión, alta pendiente, reconectar y comprobar Swagger. | Cache inicial, Swagger y producto de prueba. |
| 7-8 min | Misma funcionalidad como admin en Android e iOS; mostrar última actualización. | Emulador y dispositivo/simulador preparados. |

Preparar backend encendido, datos semilla, sesión cerrada, usuarios/roles disponibles y ejemplos de error antes de iniciar el cronómetro. En modo offline solo se exponen Inicio/Productos; las altas quedan locales hasta validar JWT/permisos al reconectar. Si el servidor responde 401, se borra el perfil local y se solicita login.

## Producto 5. Estado de entrega y evidencias

| ELEMENTO | ESTADO | SIGUIENTE ACCIÓN |
|---|---|---|
| Código y pruebas | Implementado; 37 pruebas host pasan y compilan Android/iOS compartido. | Ejecutar escenarios visuales antes de presentar. |
| Rama y commits | feature/offline-first-guillen; 71d2f2a y ee75e41, local. | Publicar en GitHub solo si se requiere la entrega remota. |
| Android / Swagger | APK compilado; recorrido real y backend no disponibles en esta sesión. | Abrir emulador, sincronizar y verificar id positivo en Swagger. |
| iOS | Código compartido compila para iosSimulatorArm64; UI no ejecutada. | Ejecutar en macOS/Xcode y adjuntar captura auténtica. |
| Informe | Este PDF consolida las sesiones 10 y 11 y actualiza la fecha al 25/10/2026. | Completar anexos visuales tras la ejecución manual. |